

Relazione del Primo Assignment di Programmazione
Concorrente e Distribuita
Pool Game

De Leonardis Federico
Lotti Irene Sofia
Zanchini Margherita

3 ottobre 2026

Indice

1	Analisi del Problema	3
2	Architettura e Design del Sistema	3
2.1	Approccio Concorrente	3
2.1.1	Versione 1: Architettura a Thread e Barriere	3
2.1.2	Versione 2: Architettura a Task ed Executor	4
3	Modellazione del Comportamento	5
4	Test di Prestazione	6
5	Verifica Formale tramite Model Checking (JPF)	8

1 Analisi del Problema

Il gioco consiste nella simulazione di un tavolo da biliardo, con diverse palle piccole e due palle più grandi, che rappresentano rispettivamente una palla controllata da un giocatore umano e una palla controllata dal computer (bot).

Il problema del programma assegnato è dato dalla drastica diminuzione delle prestazioni all'aumentare del numero di palle presenti sul tavolo. L'obiettivo del progetto è quindi quello di gestire in maniera corretta il parallelismo: delegare il calcolo delle collisioni ai Worker, ovvero thread che eseguono operazioni in maniera concorrente.

2 Architettura e Design del Sistema

Per soddisfare i requisiti della consegna, abbiamo inizialmente implementato una versione che segue il pattern MVC (Model-View-Controller). La seguente sezione descrive la modellazione del programma in modo sequenziale, senza ricorrere alla parallelizzazione del motore di calcolo.

Le palle sono modellate mediante un'interfaccia ed estesa dalle tre tipologie principali: BotBall, PlayerBall e SmallBall. Ognuna di loro possiede un metodo statico per risolvere le collisioni durante il gioco. Il tavolo è rappresentato dalla Board che aggiorna lo stato e la logica di gioco, aggiornando la posizione delle palle e gestendo le collisioni. I buchi sono state modellati in maniera simile alle palle, ma con posizione fissa e stazionaria. Quando una palla collide con un buco, viene cambiato il punteggio e la palla viene rimossa dal set. Il progresso e le statistiche di gioco sono mantenuti in una classe GameState, che contiene il punteggio ed una variabile che definisce il vincitore della partita.

La View gestisce l'interfaccia grafica tramite la libreria Java Swing. Il ViewFrame renderizza a schermo lo stato attuale del gioco disegnando le palle. La classe ViewModel, invece, permette la comunicazione tra Model e View.

Il Controller regola i frame calcolando il delta-time per garantire la fluidità del gioco. Inoltre, raccoglie gli input dell'utente e genera la logica dei movimenti del Bot.

2.1 Approccio Concorrente

2.1.1 Versione 1: Architettura a Thread e Barriere

La prima versione concorrente istanzia un numero fisso di Thread (Worker) in base al numero di core della CPU.

```
this.numThreads = Math.min(this.balls.size(),  
    Runtime.getRuntime().availableProcessors());
```

Ogni Thread prende in input un range dell'oggetto SmallBall, su cui esegue i controlli relativi alle collisioni tra tutte le palle, la biglia del giocatore e quella del bot. I Thread rimangono attivi per l'intera durata della partita.

Abbiamo gestito la sincronizzazione delle palline usando le keyword `synchronized` per prevenire problemi di concorrenza. La seguente porzione di codice mostra la logica per evitare calcoli ridondanti e ridurre conflitti di accesso.

```
for (int i = this.firstIndex; i <= this.lastIndex; i++) {
    SmallBall b1 = this.balls.get(i);
    if(!b1.isInHole()) {
        //resolve collision
        for (int j = i + 1; j < this.balls.size(); j++) {

            SmallBall b2 = this.balls.get(j);
            if(!b2.isInHole()) {
                synchronized (b1) {
                    synchronized (b2) {
                        AbstractBall.resolveCollision(b1, b2);
                    }
                }
            }
        }
    }
}
```

Per garantire che nessun Thread avanzi al frame successivo mentre gli altri stanno ancora calcolando il frame corrente, è stata implementata una barriera di sincronizzazione. La classe `Barrier` agisce come un `Monitor`, e attende che tutti i `Worker` abbiano finito il lavoro tramite i metodi `wait()` e `notifyAll()`, che servono rispettivamente per attendere e risvegliare i `Thread`.

2.1.2 Versione 2: Architettura a Task ed Executor

Nella seconda versione siamo passati da un approccio basato sui `Thread` ad un'architettura orientata ai `Task`. In questa versione i `Thread` non possiedono più un ciclo infinito. Nella classe `Board` viene aggiunto un `ExecutorService`, configurato come un `FixedThreadPool` (dimensionato sul numero di core della CPU).

La `Board` istanzia i *CollisionTask*, i quali implementano l'interfaccia `Runnable`. Il numero di task è uguale al numero di `Worker`, e ogni *CollisionTask* ha in input il proprio chunk di palline da elaborare. Il vantaggio di questa versione è l'eliminazione della necessità di usare le barriere. Infatti, l'`Executor` restituisce una `Future` per ogni `Task`, e la `Board` le raccoglie in una lista.

```
Future<?> res = executor.submit(
    new CollisionTask(start, end-1, balls, botBall, playerBall));
results.add(res);
```

Subito dopo, viene eseguito un ciclo che richiama il metodo bloccante `get()` per ogni `Future`. Il metodo attende che i risultati siano pronti e successivamente ne recupera il risultato.

```
for (Future<?> res : results) {
    try {
```

```

        res.get();
    } catch (Exception e) {
        throw new RuntimeException(e);
    }
}

```

3 Modellazione del Comportamento

Per dimostrare il corretto comportamento dell'architettura concorrente descritta nella prima versione, abbiamo scelto di modellare tramite Rete di Petri il meccanismo di sincronizzazione con barriera. Per semplicità, sono stati rappresentati quattro Worker ($W1...W4$). La transizione `WorkPermission` preleva i token dal buffer e li distribuisce simultaneamente ai Worker, che eseguono in modo asincrono e parallelo il calcolo delle collisioni sul proprio range di paline assegnato. Una volta finito il proprio lavoro, il token viene spostato nel corrispondente stato di attesa, che modella la sospensione del thread tramite il metodo `wait()`. Quando tutti e quattro i posti "Waiting" contengono un token, viene abilitata la successiva transizione e il ciclo riparte.

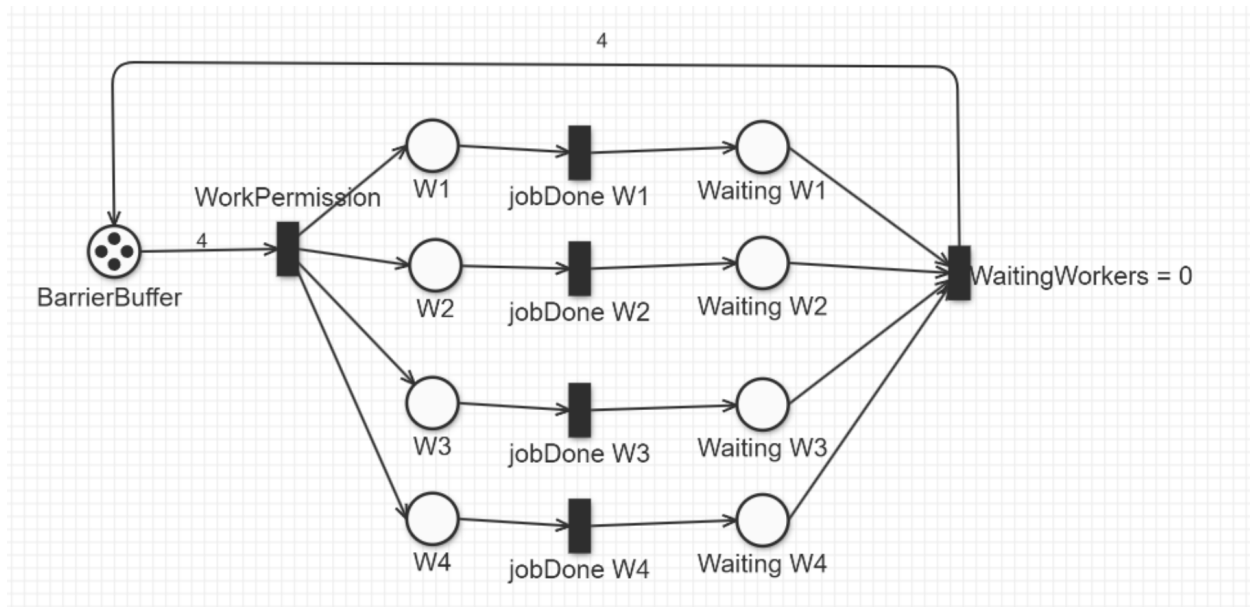


Figura 1: Funzionamento generale del rapporto tra barrier e workers.

La seconda Rete di Petri che abbiamo modellato scende a un livello di astrazione più basso per mostrare il funzionamento dell'accesso concorrente alla funzione `resolveCollision`. La rete modella lo scenario di due Worker che competono per l'acquisizione della stessa risorsa in comune (rappresentata da `smallBall`).

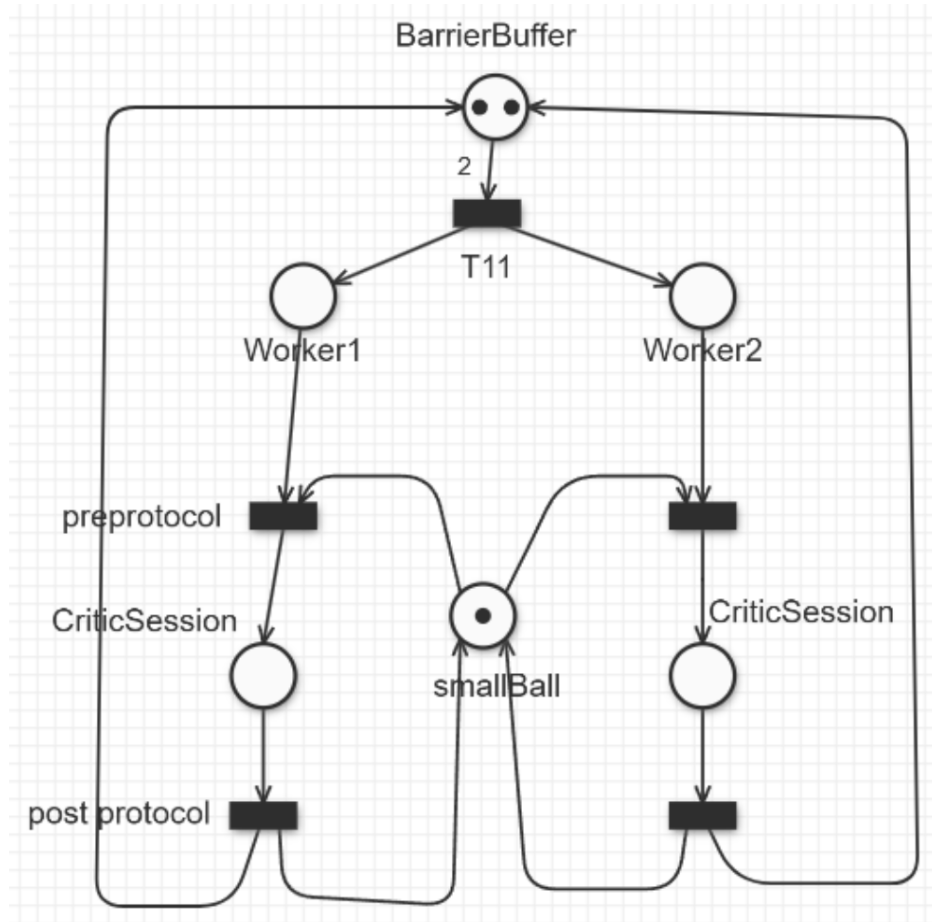


Figura 2: Comportamento di mutua esclusione semplificato per gestire l'accesso di worker multipli nel caso della smallBall

4 Test di Prestazione

Le seguenti immagini mostrano la differenza degli FPS durante la partita con 4500 palline.

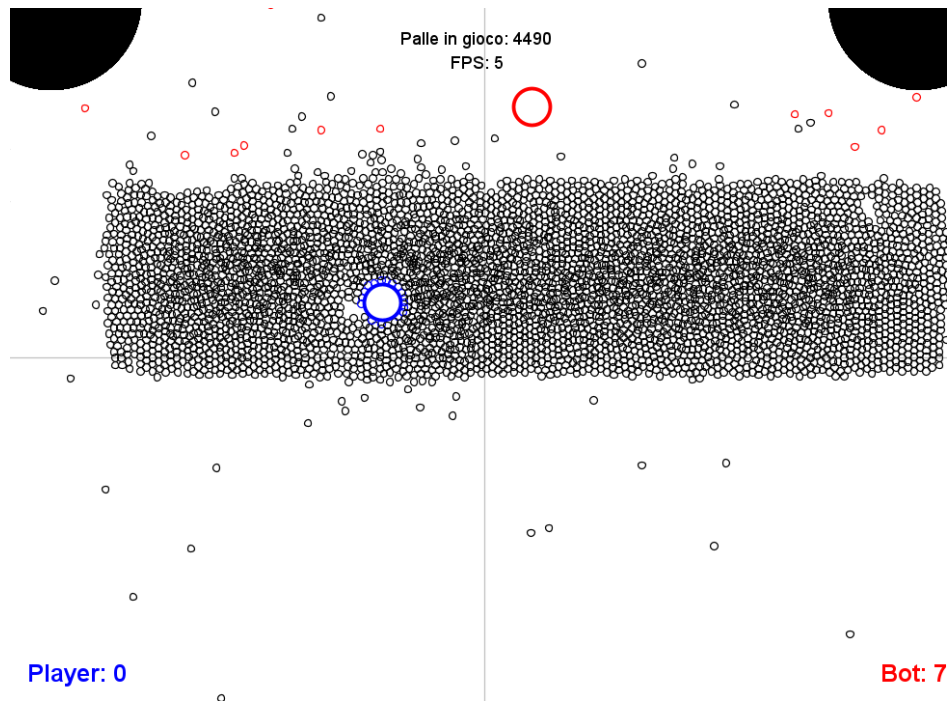


Figura 3: Versione sequenziale di gioco, FPS = 5

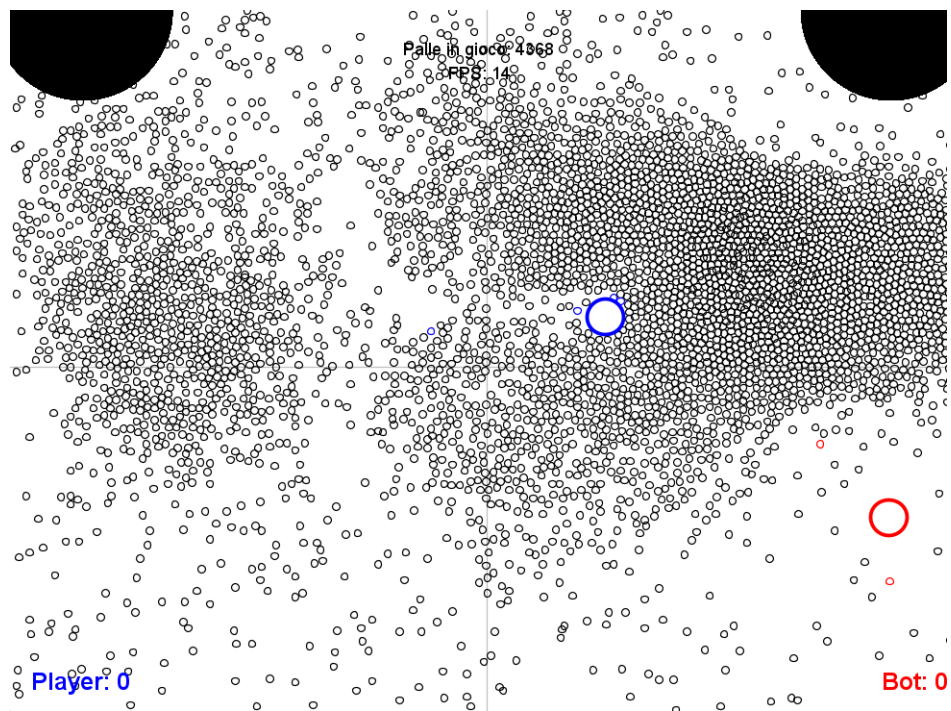


Figura 4: Versione thread di gioco, FPS = 14

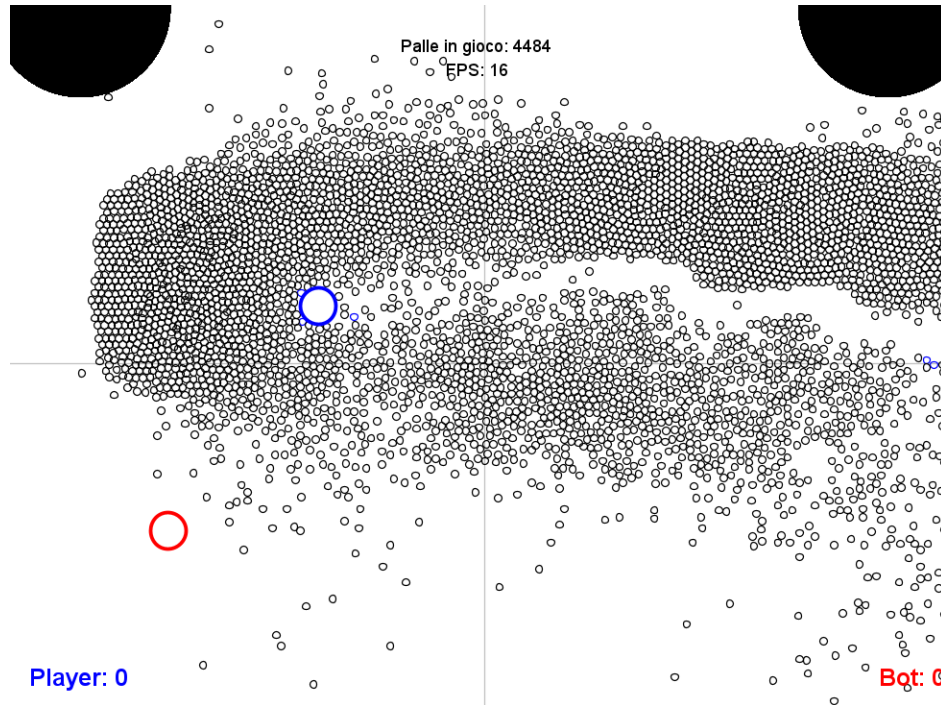


Figura 5: Versione task di gioco, FPS = 16

5 Verifica Formale tramite Model Checking (JPF)

Per verificare la correttezza del programma, abbiamo utilizzato Java Pathfinder, un framework per il model checking e la verifica di programmi concorrenti. Tramite questo tool, abbiamo potuto esplorare tutto l'albero degli stati possibili del nostro programma.

Abbiamo deciso di testare le race condition presenti nel controllo delle collisioni. Aniché eseguire l'intero programma, ci siamo concentrati sulle porzioni critiche di esso. Abbiamo aggiunto:

- Quattro SmallBall, aggiunte in un ArrayList;
- Le palline che rappresentano quella del Bot e quella del Player;
- Una barriera;
- Due worker, ognuno dei quali gestisce la collisione di due delle quattro palline create.

A questo punto, facciamo partire il metodo run dei worker, dove è presente la sezione critica. JPF ne esplora tutti gli interleaving temporali.

Al termine dell'esecuzione, il JPF restituisce un report riassuntivo, al termine del quale conferma l'assenza di errori, in particolare l'assenza di Deadlock e la presenza della mutua esclusione.

```

===== snapshot
thread java.lang.Thread:{id:0,name:main,status:RUNNING,priority:5,isDaemon:false,lockCount:0,suspendCount:0}
  call stack:
    at java.lang.Thread.start(Thread.java)
    at pcd.ass01.thread_version.testJpf.TestCollision.main(TestCollision.java:38)

thread pcd.ass01.thread_version.model.workers.Worker:{id:1,name:Thread-1,status:RUNNING,priority:5,isDaemon:false,lockCount:0,suspendCount:0}
  owned locks:pcd.ass01.thread_version.model.ball.SmallBall@223,pcd.ass01.thread_version.model.ball.SmallBall@22f
  call stack:
    at pcd.ass01.thread_version.model.ball.AbstractBall.resolveCollision(AbstractBall.java:84)
    at pcd.ass01.thread_version.model.workers.Worker.run(Worker.java:46)

===== results
no errors detected

===== statistics
elapsed time:      00:00:00
states:           new=211,visited=247,backtracked=458,end=0
search:           maxDepth=15,constraints=65
choice generators: thread=184 (signal=0,lock=15,sharedRef=132,threadApi=3,reschedule=34), data=0
heap:             new=657,released=31,maxLive=602,gcCycles=458
instructions:     15958
max memory:       124MB
loaded code:      classes=106,methods=2654

===== search finished: 9/17/26, 3:47 PM

```

Figura 6: Risultato di JPF